

HisabBot

A WhatsApp Bookkeeping Assistant for Pakistani Kirana Shops

Voice-first, multilingual, self-improving — meet shopkeepers where they already are.

Stage	Live, in beta
Live URL	hisabbot-production-aa5a.up.railway.app
Tech	FastAPI · Postgres · Claude · Whisper · WhatsApp
Languages	Urdu · Roman-Urdu · English
Target unit cost	≈ PKR 30 / shop / month
Target price	PKR 299 / shop / month

Project status snapshot · May 2026

Author: Sumair · Built with Claude Code · Repository: github.com/sumair95/hisabbot

Executive Summary

HisabBot is a WhatsApp-native bookkeeping assistant for Pakistan's 4 million-plus kirana (general store) shopkeepers. They speak to it the way they speak to a clerk — in Urdu, Roman-Urdu, or English, by voice or by text — and it records every sale, credit, and payment, tracks per-customer balances, and sends a complete daily summary at 10 PM.

There is no app to install, no buttons to press, no English forms to fill. The shopkeeper opens WhatsApp — which is already on their phone and which they already use every day — and starts talking. The bot understands.

What is built and live today

- End-to-end voice + text bookkeeping over WhatsApp Cloud API.
- Customer/supplier ledgers with running balances, derived live from a single source of truth (no double-counting).
- Daily 10 PM summary in the shopkeeper's chosen language.
- Voice notes pre-cleaned (noise reduction, loudness normalization, silence trim) before transcription, with rejection of unintelligible recordings before any API spend.
- Self-improving speech recognition: each shop's vocabulary biases every future transcription, getting more accurate with use.
- Smart correction flow with a 60-second window: shopkeepers can edit or delete a recent entry naturally without hand-holding.
- Production-grade observability: structured logs, idempotent webhook handling, dead-letter queue for failed messages, full audit trail of every inbound and outbound message.

Why this wins

Existing options for kirana bookkeeping are either **too expensive and complex** (full point-of-sale systems starting at PKR 1,500+ a month, requiring a tablet, training, and an English-comfortable operator) or **too primitive** (a paper rok-bahi that gets lost, miscalculated, or simply forgotten). HisabBot meets the shopkeeper at **their** existing tool, in **their** language, with **their** natural input mode (speaking). Marginal cost stays under PKR 30 a month per shop while supporting unlimited transactions.

The Problem

Pakistan has more than four million kirana shops — small, family-run general stores selling everyday groceries, household goods, and snacks. They are the backbone of urban and semi-urban retail, and almost every one of them runs on informal credit (udhaar) extended to regular customers.

How shopkeepers currently track money

- bullet **Paper ledger (rok-bahi)** — handwritten, language-mixed, sometimes lost or destroyed.
- bullet **Mental memory** — the shopkeeper remembers who owes what; disputes happen.
- bullet **WhatsApp messages to themselves** — informal notes that never aggregate into a real summary.
- bullet **Spreadsheets** (rare) — only the most digitally-fluent shopkeepers, and even they don't fill them in consistently.

Why off-the-shelf tools haven't worked

- bullet **POS systems** require dedicated hardware (tablet/PC), training, English-language UIs, and PKR 1,000–3,000+ monthly fees.
- bullet **Accounting apps** assume the user can read English interfaces and type with confidence.
- bullet **Voice assistants** built abroad (Alexa, Google) don't understand Roman-Urdu code-mixed speech and aren't designed for transactional bookkeeping.
- bullet **Existing local apps** are often little more than a digital ledger — they replace the rok-bahi but don't reduce the cognitive load of remembering, calculating, or summarizing.

Three observations that shaped HisabBot

- 1. WhatsApp is universal.** Every shopkeeper has it, uses it daily, and trusts it. Nothing has higher penetration in this segment.
- 2. Voice beats typing.** Shopkeepers speak Urdu/Roman-Urdu comfortably; many would rather not type, especially in a busy shop. A 5-second voice note is faster than typing the same transaction.
- 3. The bot must be a clerk, not a form.** Successful digital tools in this market mimic an existing relationship (a trusted clerk, an accountant) rather than asking the user to learn a new abstract interface.

Our Solution: HisabBot

HisabBot is a WhatsApp number the shopkeeper messages. They send a quick voice note or text in whatever language and phrasing comes naturally. The bot listens, understands, and writes the transaction to a private ledger — then confirms back in the same language.

A typical day with HisabBot

Morning, 9:30 AM

Shopkeeper (voice note): *“Ahmed bhai ko 500 ka udhaar diya, 2 kilo cheeni”*

Bot: ■ Ahmed ko PKR 500 udhaar.

■ cheeni 2kg

Kul udhaar Ahmed ka: PKR 1,200

Afternoon, 3:15 PM

Shopkeeper (text): *“450 ki coke ki bottle ki sale hoye”*

Bot: ■ Cash sale PKR 450.

■ coke 1 bottle — PKR 450

Evening, 6:00 PM

Shopkeeper (voice): *“Bilal se 300 wapas mile”*

Bot: ■ Bilal se PKR 300 received.

Bilal ka baqi udhaar: PKR 0 — hisaab clear ho gaya. ■

10:00 PM — automatic daily summary

■ Aaj ka Hisaab — 06 May 2026

■ Cash Sales: PKR 8,750

■ Udhaar Sales: PKR 3,400

■ Paisay Mile: PKR 2,100

■ Supplier ko Diye: PKR 4,500

■ Udhaar Lene Wale (Top): Ahmed (1,200), Khalid (800)...

Kul Sales: PKR 12,150

What the shopkeeper does NOT have to do

- bullet install an app or sign up on a website
- bullet Read English forms or fill structured fields
- bullet Remember exact phrasings or commands
- bullet Manually total daily sales
- bullet Jot up customer balances on paper
- bullet Use a ledger when a notebook gets damaged

How It Works

Every WhatsApp message — text or voice — flows through the same pipeline. Below is the journey of a single voice note, from the shopkeeper's lips to a row in the database.

The voice-note pipeline

- 1) WhatsApp delivers the voice note (OGG/Opus audio) to our webhook on Railway.
- 2) The bot fetches the audio bytes from WhatsApp, in parallel with looking up the shopkeeper in the database.
- 3) The audio is cleaned: a high-pass filter cuts low-frequency rumble (fans, traffic), an FFT denoiser suppresses background noise, loudness normalization brings quiet voices to broadcast level, and silence is trimmed from start and end.
- 4) Voice Activity Detection (VAD) checks whether there is any actual speech in the cleaned audio. Empty or pure-noise recordings are rejected immediately — no API spend, polite reply asking for a re-record.
- 5) The bot fetches this shop's vocabulary list (its known customers, suppliers, and recent products) and feeds it to Whisper as biasing context. Whisper produces stable, accurate transcripts of proper nouns it would otherwise mis-spell.
- 6) Whisper returns the transcript along with confidence metrics. If Whisper is uncertain, the transcript is rejected and the bot asks for a clearer recording.
- 7) The transcript goes to Claude Haiku 4.5, an LLM, which classifies the intent (TRANSACTION, QUERY, REMINDER, CORRECTION, SETTINGS_CHANGE...) and extracts structured fields — amount, customer name, items.
- 8) The orchestrator resolves customer names against the existing contact list using fuzzy matching, writes the transaction to Postgres, and computes the new balance.
- 9) A receipt-style confirmation is sent back to the shopkeeper in their preferred language.

Total round-trip: roughly 2–3 seconds for a clean voice note.

Architecture diagram

Shopkeeper (WhatsApp)

↓ voice / text

WhatsApp Cloud API webhook

↓

FastAPI on Railway — signature check, dedup, async dispatch

↓

Audio preprocessing (ffmpeg + webrtcvad) — clean and validate

↓

Whisper-large-v3 on Groq — STT with per-shop biasing

↓

Claude Haiku 4.5 (Anthropic) — intent classification + field extraction

↓

Orchestrator — fuzzy contact match, business rules, state machine

↓

Postgres on Supabase — single source of truth, view-derived balances

↓

Reply via WhatsApp Cloud API

The Build Journey

Every change below is committed to git and documented in WORKLOG.md and CHANGELOG.md inside the repository. Versions run from 0.1.0 (initial scaffold) through 0.4.0 (current).

Date	Milestone	What it means
2026-04-20	Project scaffold	FastAPI app, Postgres schema, fuzzy name matcher, prompt design, Roman-Urdu/Urdu/English reply templates. 21 unit tests.
2026-04-21	Production deployment	Live on Railway. Supabase schema applied. WhatsApp webhook registered with Meta. End-to-end test passed.
2026-04-30	STT migration to Groq	Switched speech-to-text from OpenAI's older Whisper model to Groq's whisper-large-v3 — about 3× cheaper, faster, and more accurate on Urdu.
2026-05-01	Daily summary correctness fix	Bottom-line total previously netted borrower repayments and supplier payments into the headline figure. Changed to show sales-only total; per-customer payments stay as line items.
2026-05-02	Shop-name rename via LLM intent	Initial keyword approach failed on voice transcripts. Added a new SETTINGS_CHANGE intent to the LLM extractor; both single-message ("naam Ahmed Store rakh do") and two-step ("naam galat hai" → bot asks for new name) flows now work.
2026-05-04	Per-shop Whisper vocabulary biasing	Each shop's customer/supplier/product list is fed to Whisper as biasing context on every call. Self-improving by design — the bot transcribes more accurately as the shopkeeper logs more transactions.
2026-05-04	Quieter receipts + 60-second correction window	Removed redundant daily-total chatter from per-transaction replies. Vague "galat hai" / "undo" never auto-deletes anymore — bot asks delete-or-change. Multi-message disambiguation when several entries are recent.
2026-05-05	STT noise resilience pipeline	Audio cleaning (ffmpeg: high-pass, FFT denoise, loudness normalization, silence trim), Voice Activity Detection, confidence-based rejection. Handles loud kirana environments gracefully.

Key Innovations

Voice-first, multilingual by default

Shopkeepers don't have to learn the bot's language — the bot learns theirs. Urdu script, Roman-Urdu, English, and natural code-mixing all work. The bot replies in whichever language the shopkeeper has set; switching is a one-word command.

Self-improving speech recognition

On every voice call, Whisper receives a biasing prompt built live from this shop's known customers, suppliers, and recent products. As the ledger grows, transcription gets more accurate for proper nouns. New customers added today appear in the prompt for tomorrow's voice notes — automatically.

LLM intent classification, not keyword matching

Early versions tried regex-style keyword detection for commands like "change shop name". Voice transcripts vary too much for keywords to keep up. We replaced that with explicit LLM intents (TRANSACTION, QUERY, CORRECTION, SETTINGS_CHANGE) and few-shot examples. The bot now understands natural phrasing in any of the three languages.

Recency-scoped corrections

When a shopkeeper says "galat hai" (it's wrong), the bot never auto-deletes. It looks at transactions from the last 60 seconds, asks delete-or-change, and disambiguates if multiple recent entries exist. Old entries can still be edited with specific references ("Ahmed wala 600 nahi 500 tha").

Noise-resilient audio pipeline

Kirana shops are noisy. Every voice note is cleaned by an ffmpeg pipeline — high-pass filter, FFT denoiser, loudness normalization, silence trim — before transcription. A separate Voice Activity Detection step rejects pocket recordings and pure-noise audio before any API call is made.

Confidence-aware transcription

Whisper itself returns confidence metrics on every call. We refuse low-confidence transcripts (so they never reach the LLM) and ask the shopkeeper to record again with less background noise — a safer outcome than logging garbage.

Cost-optimized model stack

Claude Haiku for primary extraction with prompt caching (~10× cheaper on cached prefix), OpenAI gpt-4o-mini as a resilience fallback, Groq's Whisper-large-v3 for STT (3× cheaper than OpenAI's hosted Whisper). Every component was chosen on a per-cent-per-message basis.

Production-grade audit trail

Every inbound and outbound message is logged with the full extraction JSON, the linked transaction (if any), and the language detected. Failed messages go to a dead-letter table. A WhatsApp message ID dedupe prevents double-billing on retried webhooks. Soft-delete only — no transaction is ever hard-deleted.

Technical Architecture

Each layer was chosen to optimise for one of: cost, developer ergonomics, or Pakistan-market suitability. The overall principle is to use boring, mature tooling wherever possible and reserve the novel choices for places where they unlock real wins (LLM-based extraction, voice-as-input, self-improving vocabulary biasing).

Layer	Technology	Why this choice
Messaging	WhatsApp Cloud API	Universal reach in target market; official Meta API; no unofficial workarounds.
HTTP / Webhook	FastAPI	Async by default, low latency, cleanest Python framework for webhooks; native Pydantic validation.
Database	Postgres on Supabase	Battle-tested. Balances are derived from a Postgres view (single source of truth). <code>pg_trgm</code> for fuzzy matching.
DB driver	asyncpg + raw SQL	No ORM. Queries are few and read more clearly as SQL than as query-builder calls.
LLM (extraction)	Anthropic Claude Haiku 4.5	Strong Urdu/Roman-Urdu performance, cost-effective with prompt caching enabled.
LLM (fallback)	OpenAI gpt-4o-mini	If Anthropic fails or rate-limits, extraction falls back here automatically. Same JSON contract.
TTS	OpenAI tts-1 (voice replies)	Optional voice replies for shopkeepers who prefer audio.
STT	Groq whisper-large-v3	OpenAI-compatible API, ~3× cheaper than OpenAI's hosted Whisper, 5× faster on Groq's LPU hardware.
Audio preprocessing	ffmpeg + webrtcvad	Standard tools, well-understood; ffmpeg's <code>loudnorm</code> and <code>aftdn</code> are production-grade for noise reduction.
Fuzzy matching	rapidfuzz	Pure-Python WRatio scoring — handles Roman-Urdu name spelling variants without needing embeddings.
Scheduling	APScheduler	Daily 10 PM summary. In-process, no external scheduler required.

Reliability and observability

- bullet **Idempotency:** every WhatsApp message has an ID; we store IDs we have already processed and ignore duplicates (Meta retries are common).
- bullet **Signature verification:** X-Hub-Signature-256 HMAC is validated on every inbound call; bad signatures are rejected before processing.
- bullet **Dead-letter queue:** any message whose handler fails is saved to a `failed\_messages` table with the original payload and error, so nothing is silently lost.
- bullet **Soft delete:** transactions are never hard-deleted — the audit trail is permanent.
- bullet **Structured logging:** JSON logs in production carry every key field (shop ID, message ID, intent, confidence metrics, latency).
- bullet **Health checks:** `/healthz` lists missing configuration and is hit by Railway's deploy verification.

Cost Economics

Unit economics are healthy from day one, with significant room to compress as we move down the model and infrastructure price curves.

Cost driver	Per-message	At 10k msg/month/shop	Notes
LLM (extraction)	≈ PKR 0.40	≈ PKR 4,000	Cached system prompt drops cost ~10× on cache hits.
STT (Groq Whisper)	≈ PKR 0.10 / voice min	≈ PKR 100 (avg shop)	Most messages are text; voice averages ~30s/day.
Database (Supabase)	—	≈ PKR 1,500 / month base	Shared across all shops; per-shop incremental ≈ 0.
WhatsApp Cloud API	Free for service msgs	—	Pay only for marketing-template-initiated convos (we don't).
Hosting (Railway)	—	≈ PKR 1,500 / month base	Single FastAPI container; scales horizontally if needed.
Total marginal /shop	—	≈ PKR 30 / month	Per-shop infra cost at 100-shop fleet, with caching.

Pricing model

- **Basic tier:** PKR 299 / shop / month — unlimited messages, daily summary, voice replies, all current features.
- **Free trial:** 14 days, no credit card.
- **Future tiers (post-launch):** per-customer reminders, inventory tracking, multi-employee, weekly/monthly P&L.;

Why the margin holds at scale

Anthropic prompt caching makes the per-call LLM cost almost flat in shop count. STT cost is bounded by voice-minute usage, which has a natural ceiling per shopkeeper (no one sends thousands of voice notes a day). Database and hosting costs amortise across the entire fleet. At 1,000 shops we expect marginal cost to drop from PKR 30 to roughly PKR 12.

Roadmap

Near-term (next 2–4 weeks)

- bullet **STT phase 4:** A/B test Whisper-large-v3-turbo on Groq (~2× faster). If accuracy holds, ship for ~600 ms latency reduction per voice note.
- bullet **STT phase 5:** Phonetic post-processing — catalog the systematic Whisper errors observed in production traffic and apply targeted fixes after STT, before LLM.
- bullet **Two-contact disambiguation:** the bot already asks "kaun sa Ahmed?" when fuzzy matching is ambiguous; refine the threshold based on real shop data.
- bullet **Customer reminders:** shopkeeper says "Ahmed ko kal 500 yaad dilana"; bot pings them at the chosen date. Schema and intent already in place; needs cron wiring.

Medium-term (1–3 months)

- bullet **Self-improving phase 2 — dynamic LLM few-shot:** maintain a rolling buffer of each shop's recent successful extractions and inject them as additional examples in the LLM prompt. Adapts the bot to each shopkeeper's individual phrasing.
- bullet **Self-improving phase 3 — alias dictionary:** when a shopkeeper does fix_customer ("Ahmed nahi Ali tha"), log the (heard, correct) pair. Use it as both a Whisper biasing hint and an LLM extraction rule.
- bullet **Weekly and monthly summaries** on demand.
- bullet **Customer-facing reminders** sent from the shopkeeper's number with one-tap approval (highest-value follow-on).

Longer-term (3–9 months)

- bullet **Inventory tracking** — link sales to a stock ledger; alert on low stock.
- bullet **Photo-of-receipt extraction** — shopkeeper sends a supplier invoice as an image; bot extracts line items.
- bullet **Multi-employee per shop** — shop owner assigns one or more numbers as authorised data-entry phones.
- bullet **Bank/EasyPaisa integration** — auto-reconcile inbound payments against credit ledger.

Explicitly NOT in scope (yet)

Inventory management, full POS, web/mobile dashboards, automated invoicing, and real-time payment processing are each multi-month rabbit holes that don't move the wedge for our first 1,000 shops. We earn the right to build them by nailing the bookkeeping core first.

Why HisabBot Wins

Most digital-bookkeeping efforts in this market die at the adoption stage — the shopkeeper either can't or won't switch behaviour. HisabBot wins by not asking them to. Same WhatsApp, same language, same way they already speak about money. The product is the conversation.

	Paper rokh-bahi	Local POS apps	Generic accounting	HisabBot
Cost / month	PKR 0	PKR 1,000–3,000+	PKR 1,500+	PKR 299
Hardware needed	Pen + book	Tablet/PC	PC	Already on phone
English required	No	Yes	Yes	No
Voice input	—	Rare	No	Yes (default)
Setup time	0	Hours/days	Days	1 minute
Lost-data risk	High	Low	Low	Low
Daily summary	Manual	Yes	Yes	Automatic, voice
Customer balance	Memory	Yes	Yes	Always live

Defensibility

Network effects of vocabulary: the more transactions a shop logs, the more accurate the bot becomes for that specific shop. A new entrant starts from zero and has to catch up customer by customer.

Data moat for fine-tuning: we accumulate (audio, correct-transcript) and (correction, fix) pairs over time — this becomes a private dataset for future model tuning that no competitor can replicate cheaply.

Distribution lock-in: WhatsApp is the entry point; switching costs for the shopkeeper are real (re-onboarding all customers, history loss).

Glossary — Plain-English Definitions

For readers without a software background. Skip if you've seen these terms before.

API	Application Programming Interface. The standard way two software systems talk to each other over the network.
Cloud	A remote server you rent rather than own.
FastAPI	The Python web framework HisabBot's server is built on.
Few-shot examples	Sample input/output pairs included in an AI prompt to teach the model the desired behaviour without retraining it.
ffmpeg	The standard open-source tool for processing audio and video. HisabBot uses it to clean voice-note audio.
Fuzzy matching	Matching text that's close-enough rather than exactly equal — e.g. recognising "Ahmad" and "Ahmed" as the same person.
Groq	A company that runs AI models on custom hardware called LPUs (Language Processing Units), much faster than typical GPUs.
Idempotent	An operation that can be repeated safely without changing the outcome — important so retried webhook calls don't double-charge anyone.
Intent	The high-level meaning of a message — "is this a sale, a query, a correction?"
Kirana	Pakistan/India term for a small general/grocery store.
LLM	Large Language Model — the kind of AI that powers ChatGPT and Claude. HisabBot uses one to understand what shopkeepers mean.
LPU	Language Processing Unit — Groq's custom AI hardware, optimised for speed.
Postgres	An open-source database. HisabBot's transactions, customers, and shopkeeper records live here.
POS	Point of Sale — the cash-register-and-software system used in modern retail.
Prompt caching	An Anthropic feature that stores a long unchanging prompt prefix on their side, making repeated calls ~10× cheaper.
rok-bahi (ꣳꣳꣳꣳ ꣳꣳꣳꣳ)	The traditional handwritten ledger used by Pakistani shopkeepers.

STT	Speech-to-Text — the technology that turns spoken words into written text. Whisper is the model HisabBot uses.
Soft delete	Marking a database row as "deleted" without actually erasing it. Lets you undo and audit later.
Supabase	A managed Postgres database service. HisabBot's data lives on their infrastructure.
TTS	Text-to-Speech — turning written text into spoken audio. Used for voice replies.
udhaar (■■■■■)	Credit extended to a customer — they take goods now and pay later.
VAD	Voice Activity Detection — software that decides whether a given chunk of audio contains real speech or just silence/noise.
Webhook	A URL that another service calls whenever an event happens — e.g. WhatsApp calls our webhook every time a shopkeeper sends a message.
Whisper	OpenAI's open-source speech-recognition model, considered state of the art for multilingual transcription.